TECHNICAL PROJECT REPORT

Abona Shop

A full-stack e-commerce platform built for the Thai market — architecture, implementation, security, and deployment.

Hermann N'zi Ngenda

Full-Stack Web Development · April 2026

🌐 Visit Live Site — abona-3ve.pages.dev

Table of Contents

01 Project Overview

02 System Architecture

03 Frontend Implementation

04 Backend Implementation

05 Database Design

06 Authentication & Session Management

07 Security Layer — Arcjet

08 Payment Processing — Stripe

09 Email Notification System — Brevo

10 Admin Panel

11 Deployment Strategy

12 Challenges & Solutions

13 Conclusion

01 Project Overview

Live Site: https://abona-3ve.pages.dev

Abona Shop is a fully functional e-commerce web application designed and built from scratch for the Thai retail market. The project covers the complete scope of a modern online store — product browsing, cart management, secure checkout with real payment processing, order tracking, and a dedicated admin control panel for managing the business. The name *Abona* is paired with the tagline *Look Around You*, reflecting a brand identity centered on discovery and everyday elegance.

The project was built entirely with hand-written code — no frontend framework, no CMS, no boilerplate starter kit. Every page, every API route, every database query was written deliberately, which gave me full control over performance, security, and the user experience at every layer of the stack.

Core Features

Product Catalog

Searchable, filterable product grid with categories, star ratings, and responsive image display.

Persistent Cart

Cart saved server-side per user, survives page refreshes and device switches.

Stripe Payments

Full PaymentIntent flow in Thai Baht, supporting cards, wallets, and 3D Secure.

JWT Authentication

Secure httpOnly cookie sessions. Passwords bcrypt-hashed. XSS-safe by design.

Order Management

Full order lifecycle from placement to delivery, with status tracking and cancellation.

Wishlist

Per-user wishlist stored in the database, accessible across devices.

Reviews

One review per user per product, auto-updating product star ratings via SQL trigger.

Coupon Codes

Percentage and fixed-amount discounts with expiry, usage limits, and validation.

Dark Mode

FOUC-free dark mode toggle using CSS custom properties and localStorage.

Email Notifications

Transactional emails for orders, shipping, delivery, and password reset via Brevo.

Arcjet Security

Bot protection, rate limiting, and email validation on all auth endpoints.

Admin Panel

Separate authenticated dashboard for products, orders, users, coupons, and reviews.

Technology Choices

The decision to use vanilla HTML, CSS, and JavaScript on the frontend — rather than a framework like React or Vue — was intentional. It forced a deeper understanding of how browsers work, how the DOM updates, and how to architect modular JavaScript without external scaffolding. On the backend, Node.js with Express was chosen for its non-

blocking I/O model, which handles concurrent API calls efficiently without the overhead of a heavier framework.

- HTML / CSS / ES Modules
- Node.js + Express.js
- MySQL / MariaDB
- JWT + bcryptjs
- Stripe Payment Element
- Arcjet
- Brevo HTTP API
- Day.js

02 System Architecture

The application follows a classic three-tier architecture: a static frontend served over a CDN, a stateless REST API backend running on a managed cloud platform, and a cloud-hosted relational database. The three layers communicate exclusively over HTTPS and are deployed independently, which means each can be scaled, updated, or replaced without affecting the others.

```
┌──────────────────────────────────────────────────────────┐ │ USER BROWSER
│ │ HTML + CSS + ES Modules (no framework) │
└──────────────────────┬───────────────────────────────────┘ │ HTTPS (fetch
+ credentials: 'include') ▼
┌──────────────────────────────────────────────────────────┐ │ CLOUDFLARE
PAGES │ │ Static file delivery via global CDN edge network │
└──────────────────────────────────────────────────────────┘ │ API calls to
/api/* ▼ ┌──────────────────────────────────────────────────┐ │
RENDER — Express API │ │ ┌──────────┐ ┌────────┐ ┌────────┐ ┌──────────┐
│ │ │ Arcjet │ │ JWT │ │ Stripe │ │ Brevo │ │ │ │ Security │ │ Auth │ │
Payments │ │ Email │ │ │ └──────────┘ └────────┘ └────────┘ └──────────┘
│ └──────────────────────┬───────────────────────────────┘ │ SSL/TLS
encrypted connection ▼
┌──────────────────────────────────────────────────────────┐ │ AIVEN — MySQL
Database │ │ SOC 2 certified · Encrypted at rest · Daily backups │
└──────────────────────────────────────────────────────────┘
```

Request Lifecycle

When a user interacts with the site — adding a product to their cart, for example — the browser sends an `XMLHttpRequest` (via the `fetch` API) to the Render backend. The request carries the JWT token automatically through an httpOnly cookie. The backend middleware validates the token, queries the database, and returns JSON. The frontend then updates the DOM accordingly. No page reload is required.

Separation of Concerns

The frontend never directly touches the database. All business logic lives in the backend: stock decrements happen server-side within transactions, coupon validation happens server-side before any discount is applied, and order totals are calculated server-side regardless of what the frontend sends. This prevents any client-side manipulation of prices or inventory.

---

03 Frontend Implementation

The frontend is a collection of HTML pages each linked to their own CSS stylesheet and JavaScript module. There is no build step, no bundler, and no transpiler — the browser receives exactly what is written on disk. This approach makes the site extremely fast to load and straightforward to debug.

Page Structure

Page	Purpose
`index.html`	Product catalog with search, filters, and add-to-cart
`login.html`	Sign-in with email + password
`register.html`	Account creation
`forgot-password.html`	Triggers password reset email
`reset-password.html`	Sets new password using token from email
`product.html`	Single product detail, reviews, share button, recently viewed
`checkout.html`	Order summary, coupon, Stripe payment element
`orders.html`	Order history with status and tracking
`wishlist.html`	Saved products
`tracking.html`	Order tracking status view
`settings.html`	Account settings and address management

Shared Utilities

Three utility modules are shared across all pages:

- `scripts/utils/api.js` — exports the `API_BASE` URL constant
- `scripts/utils/money.js` — `formatCurrency()` helper for Thai Baht formatting
- `scripts/utils/darkmode.js` — toggle logic and localStorage persistence

Dark Mode Implementation

Every page includes this inline script as the very first element inside `<head>`, before any stylesheet is loaded:

```
if (localStorage.getItem('darkMode') === 'true')
  document.documentElement.classList.add('dark');
```

Because this runs synchronously before the browser parses any CSS, there is no flash of the wrong theme on page load. All dark-mode overrides use CSS custom properties scoped to `html.dark`, allowing a single class toggle to recolor the entire interface.

Product Grid

The homepage product grid uses CSS `grid` with `auto-fill` and `minmax(200px, 1fr)`, meaning it adapts fluidly from one column on mobile to five or six columns on large screens with no JavaScript involved. Product images are placed inside fixed-height containers with `object-fit: contain`, ensuring that items of every aspect ratio — portrait clothing photos, square electronics, landscape home goods — all display at a consistent size without cropping or distortion.

Recently Viewed Products

When a customer views a product page, the product's ID, name, image, and price are written to `localStorage` under the key `recently Viewed`. The list is capped at eight items, with the most recent always appearing first. On subsequent product page visits, the recently viewed strip renders below the main product, giving customers a quick path back to items they considered earlier.

04 Backend Implementation

The backend is a Node.js application using Express.js, structured around feature-based route modules. Each route file handles one resource — `auth.js` for authentication, `cart.js` for cart operations, `orders.js` for order management, and so on. This separation makes the codebase easy to navigate and ensures that a change to one resource cannot accidentally break another.

Server Entry Point

```
import express from 'express';
import cors from 'cors';
import cookieParser from 'cookie-parser';

app.use(cors({ origin: process.env.CLIENT_ORIGIN, credentials: true }));
app.use(express.json());
app.use(cookieParser());

app.use('/api/auth',     authRoutes);
app.use('/api/products', productRoutes);
app.use('/api/cart',     cartRoutes);
app.use('/api/orders',   orderRoutes);
app.use('/api/payment',  paymentRoutes);
// ... additional routes
app.use('/admin',        adminRoutes);
```

CORS is configured with `credentials: true` and an explicit `origin` matching the frontend URL. This is required for the browser to include the httpOnly JWT cookie in cross-origin API requests — without it, every authenticated call would fail.

Route Overview

Method	Endpoint	Auth	Description
POST	/api/auth/register	—	Create account

Method	Endpoint	Auth	Description
POST	/api/auth/login	—	Sign in, set JWT cookie
GET	/api/auth/me	Cookie	Get current user
POST	/api/auth/forgot-password	—	Send reset email
POST	/api/auth/reset-password	—	Set new password
GET	/api/products	—	List all products
GET	/api/products/:id	—	Single product
GET / POST	/api/cart	✓	Get / add cart items
PATCH / DELETE	/api/cart/:id	✓	Update quantity or remove item
POST	/api/orders	✓	Place order from cart
PATCH	/api/orders/:id/cancel	✓	Customer cancels order
POST	/api/payment/create-intent	✓	Create Stripe PaymentIntent
GET / POST / DELETE	/api/wishlist	✓	Wishlist management
POST	/api/coupons/validate	✓	Validate coupon code
GET / POST	/api/reviews/:productId	Partial	Read / submit reviews

Order Placement Transaction

Placing an order is the most complex backend operation and is handled inside a MySQL transaction to guarantee data consistency. If any step fails — inserting the order, inserting line items, or decrementing stock — the entire operation rolls back and the customer's cart is left untouched. No partial orders are ever written to the database.

```
await conn.beginTransaction();

// 1. Insert order header
await conn.execute('INSERT INTO orders (...) VALUES (...)', [...]);

// 2. Insert each line item
for (const item of cartItems) {
  await conn.execute('INSERT INTO order_items (...) VALUES (...)', [...]);
}

// 3. Decrement stock
await conn.execute('UPDATE products SET stock = stock - ? WHERE id = ?', [...]);

// 4. Clear the cart
await conn.execute('DELETE FROM cart_items WHERE user_id = ?', [userId]);

await conn.commit();
// Emails sent after commit — non-blocking fire-and-forget
```

05 Database Design

The database schema is designed around a central `users` table from which all transactional data — carts, orders, wishlists, reviews — radiates outward. Foreign key constraints with `ON DELETE CASCADE` ensure that removing a user account automatically removes all associated data, keeping the database clean without requiring application-level cleanup logic.

```
users ├── sessions (multi-device JWT tracking) ├── addresses (saved shipping
addresses) ├── password_resets (secure tokens, 1hr expiry) ├── cart_items
──────────┐ ├── wishlists │ │ └── wishlist_items │ products ├── reviews
──────────┼──────────────┐ └── orders └─── product_variants │ ├──
order_items product_images │ ├── order_status_logs product_categories └──
payments categories coupons └── coupon_uses
```

Products Table

The `products` table stores a denormalised `stars` and `rating_count` column alongside the live review data. Rather than computing the average rating on every product list query (which would require an expensive `JOIN` and `GROUP BY` across the reviews table), these values are maintained automatically by a MySQL trigger that fires on every review insert, update, or delete.

```sql
-- Trigger: auto-update product rating when a review changes
UPDATE products
SET
  stars        = (SELECT ROUND(AVG(stars), 2)
                  FROM reviews
                  WHERE product_id = NEW.product_id
                    AND is_approved = TRUE),
  rating_count = (SELECT COUNT(*)
                  FROM reviews
                  WHERE product_id = NEW.product_id
                    AND is_approved = TRUE)
WHERE id = NEW.product_id;
```

Order Data Snapshot

When an order is placed, product names, images, and prices are copied into the `order_items` table at that moment rather than referencing the live `products` table. This is a deliberate design choice: if an admin changes a product's price or name later, historical order records remain accurate and reflect exactly what the customer was shown and charged at the time of purchase.

Password Reset Flow

The `password_resets` table stores a `crypto.randomBytes(32)` token with a one-hour expiry. When a customer requests a reset, any existing token for that user is deleted first, then a new one is inserted. The reset URL containing the token is sent by email. On submission, the token is validated against `expires_at > NOW()` before the password is updated. After a successful reset, the token row is deleted immediately.

06 Authentication & Session Management

Authentication uses JSON Web Tokens stored in `httpOnly` cookies. The choice of httpOnly cookies over `localStorage` is security-critical: JavaScript cannot read an httpOnly cookie, which means a cross-site scripting attack that injects malicious script into the page cannot steal the user's session token. The token is sent automatically by the browser on every request to the API domain.

```
Browser Backend Database │ │ │ ├── POST /api/auth/login ──►│ │ │ { email,
password } │── SELECT user WHERE email ─►│ │ │◄── { id, hash, role } ────┤ │
│── bcrypt.compare(pw, hash) │ │ │── jwt.sign({ id, email, │ │ │ name, role },
secret) │ │◄── Set-Cookie: token=JWT │ │ │ HttpOnly; Secure; │ │ │
SameSite=Strict │ │ │ │ │ ├── GET /api/auth/me ────────►│ │ │ (cookie sent auto)
│── jwt.verify(token) │ │◄── { user object } ────────┤ │
```

Token Payload

```
// JWT payload — signed with JWT_SECRET, expires in 7 days
{
  id:    "user-uuid",
  email: "customer@example.com",
  name:  "Hermann N'zi Ngenda",
  role:  "user"  // or "admin"
}
```

Admin Authentication

The admin panel uses a completely separate JWT secret (`JWT_ADMIN_SECRET`) and a separate middleware. Even if someone obtained a customer-level JWT, it would be signed with the wrong secret and rejected by every admin route. The admin login endpoint also does not go through Arcjet's email validation rule — it uses its own separate rate limit to prevent brute-force attempts without adding friction for the legitimate admin.

Password Security

Passwords are never stored in plain text. When a user registers, the password is run through `bcryptjs.hash(password, 10)` before being written to the database. The number 10 refers to the cost factor — it means the hashing function performs 2^{10} (1,024) rounds of processing, making it computationally expensive to brute-force even if the database were compromised.

07 Security Layer — Arcjet

Arcjet sits as middleware in front of the three most sensitive auth routes — registration, login, and password reset — and applies three independent protection layers before any application code or database query runs.

```
Incoming Request │ ▼ ┌──────────────────────────────┐ │ Arcjet Shield │ │
Detects bots, headless browsers │ │ scrapers, and known attack tools │
└──────────────┬───────────────────────────┘ │ (human traffic passes) ▼
┌──────────────────────────────┐ │ Sliding Window │ │ Rate limit per IP
address: │ │ · Register → 10 req / 60 min │ │ · Login → 10 req / 15 min │ │ ·
Reset → 3 req / 15 min │ └──────────────┬───────────────────────────┘ │ (within limits)
▼ ┌──────────────────────────────┐ │ Email Validation │ │ Rejects addresses
that are: │ │ · Disposable (Mailinator etc.) │ │ · Syntactically invalid │ │
· Missing MX DNS records │ └──────────────┬───────────────────────────┘ │ (valid email)
▼ Route Handler
```

> **WHY THIS MATTERS**
>
> Credential stuffing attacks — where bots attempt thousands of username/password combinations from stolen lists — are blocked at the shield layer before a single database query is executed. The rate limiter ensures that even a human attacker trying manually cannot exceed 10 login attempts per 15 minutes per IP address.

Development Mode

In local development, Arcjet is configured with `ARCJET_ENV=development`, which relaxes some rules so that testing is not obstructed. In production on Render, this variable is changed to `production`, enabling the full enforcement of all rules.

08 Payment Processing — Stripe

Payments are handled through Stripe's Payment Element, which is a pre-built, PCI-compliant UI component that renders inside the checkout page. Stripe's servers handle all card data — the application never sees, stores, or transmits raw card numbers. This means the application is out of scope for the most burdensome PCI DSS requirements.

```
Frontend Backend Stripe │ │ │ ├─ GET /api/config ──────►│ │ │◄─ {
publishableKey } ────┤ │ │ │ │ ├─ POST /api/payment/ │ │ │ create-intent
──────────►│ │ │ { amountCents } │─ stripe.paymentIntents │ │ │ .create({ │ │
│ amount, │ │ │ currency: 'thb' │ │ │ }) ──────────────────►│ │ │◄─ {
client_secret } ──────┤ │◄─ { clientSecret } ──────┤ │ │ │ │ │
stripe.elements() │ │ │ (card UI renders) │ │ │ │ │ ├─
stripe.confirmPayment() ─┼──────────────────────►│ │ │ (3DS if needed) │
│◄─ status: 'succeeded' ──┼──────────────────────┤ │ │ │ ├─ POST
/api/orders ──────►│─ Transaction: save order │ │◄─ { orderId }
──────────┤ clear cart, update stock │
```

Currency

All amounts are processed in Thai Baht (`THB`), stored as satang (the smallest unit, equivalent to 1/100 of a Baht). An amount of ฿159.00 is stored and transmitted as the integer `15900` . This avoids any floating-point rounding issues in price calculations.

Tax Calculation

Thailand's VAT rate of 7% is applied server-side. The formula is:

```
const subtotalAfterDiscount = productTotal - discountCents;
const totalBeforeTax        = subtotalAfterDiscount + shippingTotal;
const tax                   = Math.round(totalBeforeTax * 0.07);
const total                 = totalBeforeTax + tax;
```

> **TEST CARD**

> During development and testing, use Stripe's test card number **4242 4242 4242 4242** with any future expiry date and any 3-digit CVC. No real charges are made in test mode.

---

09 Email Notification System — Brevo

Transactional emails are sent through Brevo's HTTP API rather than traditional SMTP. The distinction matters in practice: SMTP relies on TCP connections to ports 587 or 465, which are commonly blocked by university networks, corporate firewalls, and many shared hosting providers. Brevo's HTTP API sends email through standard HTTPS on port 443 — the same port as all web traffic — which is never blocked.

```
Route Handler │ ▼ sendPasswordReset() / sendOrderConfirmation() / ... │ ▼
Brevo HTTP API ─── HTTPS POST ──► api.brevo.com │ ▼ Brevo SMTP Servers │ ▼
Customer Inbox
```

Notification Matrix

Event	Customer	Admin
Order placed	✅	✅
Admin marks order as paid	✅	—
Admin marks order as shipped	✅	—
Admin marks order as delivered	✅	—
Admin cancels order	✅	—
Customer cancels order	—	✅
Product stock falls below 5	—	✅
Password reset requested	✅	—

Non-blocking Delivery

Email sending is deliberately fire-and-forget. After an order is committed to the database, the email function is called with `.catch()` but without `await`. This means a slow email delivery or a transient Brevo API error does not delay the API response to the customer or risk rolling back the order transaction. Failures are logged to the server console for monitoring.

```
// Order confirmed in DB — send email without blocking the response
sendOrderConfirmation({ to, name, orderId, items, totalCents, ... })
  .catch(err => console.error('Confirmation email failed:', err));

return res.status(201).json({ message: 'Order placed', orderId });
```

10 Admin Panel

The admin panel is served directly by the Express backend at `/admin`. It is a completely separate HTML application with its own authentication flow, using a different JWT secret from the customer-facing site. A valid customer JWT cannot grant access to the admin panel under any circumstance.

Admin Capabilities

Dashboard

Revenue overview, pending orders count, recent sales chart, and order status breakdown.

Products

Add, edit, and delete products. Manage stock levels, images, pricing, and variants.

Orders

View all orders, update status (pending → paid → shipped → delivered), and trigger customer emails.

Coupons

Create percentage and fixed-amount discount codes with expiry and usage limits.

Users

View registered customers, promote to admin, or manage account status.

Reviews

Moderate customer reviews — approve or reject before they appear publicly on product pages.

EMAIL TRIGGERS FROM ADMIN

> Every order status update made through the admin panel automatically sends the appropriate email to the customer. Marking an order as "shipped" sends a shipping notification. Marking it as "delivered" sends a delivery confirmation. Cancelling sends a cancellation notice. This happens server-side with no additional steps required from the admin.

---

11 Deployment Strategy

The three layers of the application are deployed to three separate cloud platforms, each chosen for the specific guarantees it provides for that layer. All three services have generous free tiers and production-grade infrastructure.

Layer	Platform	Why
Frontend	Cloudflare Pages	Unlimited bandwidth, global CDN, DDoS protection, free SSL
Backend	Render	Managed Node.js hosting, auto-deploy from GitHub, free tier
Database	Aiven MySQL	SOC 2 certified, SSL enforced, encrypted at rest, daily backups

Cloudflare Pages — Frontend

Cloudflare Pages serves the `frontend/` folder as a static site distributed across Cloudflare's global edge network. Because the frontend is purely static — no server-side rendering, no dynamic content — every page load is served from a data center within milliseconds of the user, regardless of where in the world they are. Cloudflare automatically provisions and renews SSL certificates and provides enterprise-grade DDoS protection at no cost.

Render — Backend

Render runs the Express API as a managed web service. Deployments are triggered automatically whenever code is pushed to the main branch on GitHub, meaning the production backend is always in sync with the latest version of the code. Environment variables — database credentials, JWT secrets, API keys — are stored securely in Render's dashboard and injected at runtime, never committed to the repository.

Aiven — Database

Aiven provides a managed MySQL service with SOC 2 Type 2 certification, meaning their security controls have been independently audited. All connections to Aiven require SSL/TLS — unencrypted connections are rejected. Data is encrypted at rest, and automated daily backups are retained for seven days, providing a recovery point in the event of accidental data loss.

Database SSL Connection

```
// backend/db/connection.js — SSL enabled for Aiven
const pool = mysql.createPool({
  host:     process.env.DB_HOST,
  port:     process.env.DB_PORT,
  user:     process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  ssl: process.env.DB_SSL === 'true' ? { rejectUnauthorized: true } : false
});
```

Production Environment Variables

```
NODE_ENV=production
CLIENT_ORIGIN=https://abona.pages.dev
DB_HOST=your-aiven-host.aivencloud.com
DB_SSL=true
ARCJET_ENV=production
STRIPE_SECRET_KEY=sk_live_...
BREVO_API_KEY=xkeysib_...
```

12 Challenges & Solutions

CORS and Cookie Authentication

The most persistent issue during development was getting cross-origin cookies to work correctly. The frontend runs on port 5500 (Live Server) and the backend on port 3000 — different ports mean different origins. By default, browsers block cookies in cross-origin requests. The solution required three things to align simultaneously: `credentials: true` in the CORS config, `credentials: 'include'` in every `fetch` call, and an exact match between the `CLIENT_ORIGIN` environment variable and the actual URL the browser was using. When the project was restructured into a `frontend/` subfolder and Live Server was reconfigured to serve from that folder directly, the origin changed from `http://127.0.0.1:5500/frontend` to `http://127.0.0.1:5500`, breaking all authenticated requests until the environment variable was updated.

SMTP Blocked on Restricted Networks

During local development on a university network, both SMTP ports 587 and 465 were blocked by the network firewall, making it impossible to send emails through Gmail's SMTP server. The solution was to replace Nodemailer entirely with Brevo's HTTP API, which sends email over standard HTTPS on port 443. This port is never blocked because it carries all regular web traffic. The change required rewriting `backend/utils/email.js` to use the `@getbrevo/brevo` SDK and updating the environment variables accordingly.

Missing Database Table

The `password_resets` table was referenced in the auth routes but had never been added to the schema file. The forgot-password endpoint was silently returning a 500 error because the SQL query was failing against a non-existent table. The table was created directly in the database and then added to `schema.sql` so that future database setups will include it from the start.

Product Image Consistency

The product catalog contains items from multiple categories — clothing, electronics, kitchen goods, sporting equipment — each with images of very different aspect ratios. Early versions of the product grid showed cards of inconsistent heights, with tall portrait photos of clothing pushing rows out of alignment. The fix was to give the image container a fixed height of 200px with `overflow: hidden` and set `object-fit: contain` on the image itself, so every product image fits within the same bounding box without cropping.

Project Restructuring on Windows

Moving the frontend assets into a `frontend/` subfolder was complicated by Windows file permission restrictions — the operating system prevented moving directories that were open in VS Code. The workaround was to copy the folders with `cp -r` rather than move them, then update the backend's static file paths and the Live Server configuration. The old root-level duplicate folders were left for manual deletion once VS Code was closed.

---

13 Conclusion

Abona Shop is a complete, production-ready e-commerce platform built without shortcuts. Every layer — from the pixel-level CSS to the database transaction logic — was written by hand, which meant confronting the real complexity of building a secure, consistent web application rather than abstracting it away behind a framework.

The most instructive parts of the project were the ones that required understanding systems at a lower level than most tutorials cover: why cookies need `SameSite` attributes and what that means for cross-origin requests, why database transactions exist and what happens when they are not used, why bcrypt's cost factor is a security parameter rather than an implementation detail, and why email delivery is not a simple problem even when the code is correct.

The deployment architecture — Cloudflare for the frontend, Render for the backend, Aiven for the database — reflects a genuine consideration of what each layer requires rather than a default choice. Cloudflare's edge network is the right fit for static assets. A managed Node.js host is the right fit for a stateless API. A SOC 2 certified database provider is the right fit for customer and payment data.

The project is ready for real customers. The Stripe integration is in test mode and can be switched to live keys when a business registration is in place. The email system is operational. The admin panel is functional. The security layer is active. What remains is populating the product catalog, verifying the sender domain in Brevo, and completing the deployment steps outlined in this report.

> **PROJECT STATUS**
> All core features are implemented and tested locally. Authentication, cart, checkout, orders, wishlist, reviews, coupons, admin panel, and email notifications are all functional. The application is ready for deployment to Cloudflare Pages, Render, and Aiven MySQL.

Hermann N'zi Ngenda

Full-Stack Web Development

April 2026

Abona Shop — Technical Report

Abona Shop · Built by Hermann N'zi Ngenda · April 2026